RDFIA 2 - c

Julien Kordes

December 2024

Table des matières

1 Introduction **2**

2 Practice **2**

2.1 Naive method 2

2.2 Domain Adverserial Neural Network 3

2.3 Model improvement 6

2.3.1 Learning rate scheduler 6

2.3.2 GRL factor scheduling 7

2.3.3 Standardization 9

2.3.4 Number of epochs 9

1 Introduction

This lab explores domain adaptation as described in Article **[1]**. To illustrate this, we will use the MNIST (FIGURE 1) dataset as the source dataset and MNIST-M (FIGURE 2) as the target dataset. The two datasets are very similar and share the numbers they represent. However, they differ in the colors and contrast used.

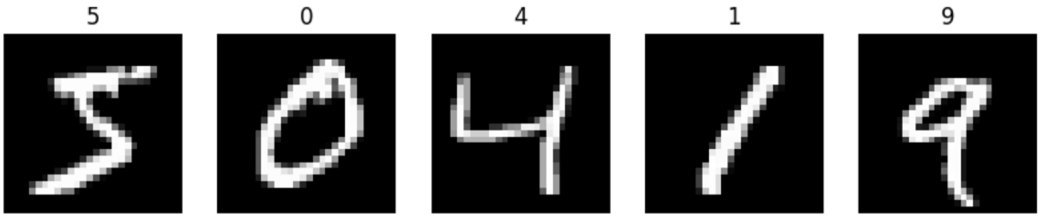

Figure 1: Visualization of MNIST Dataset

Figure 2: Visualization of MNIST-M Dataset

2 Practice

2.1 Naive method

The first neural network follows the architecture described in FIGURE 3. The initial goal is to evaluate its performance on the MNIST-M dataset after training it on the MNIST dataset.

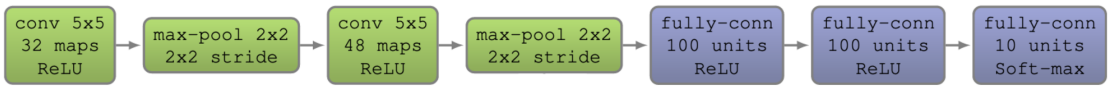

Figure 3: Architecture of the naive neural network

The FIGURE 4 shows that the model was able to train well (the loss converges really fast and is very stable) on the source dataset and reached an accuracy of **99.07**. However, the accuracy when tested on the MNIST-M dataset goes down to **52.52**. Clearly, our model is not able to generalize to the MNIST-M dataset after being only trained on the MNIST.

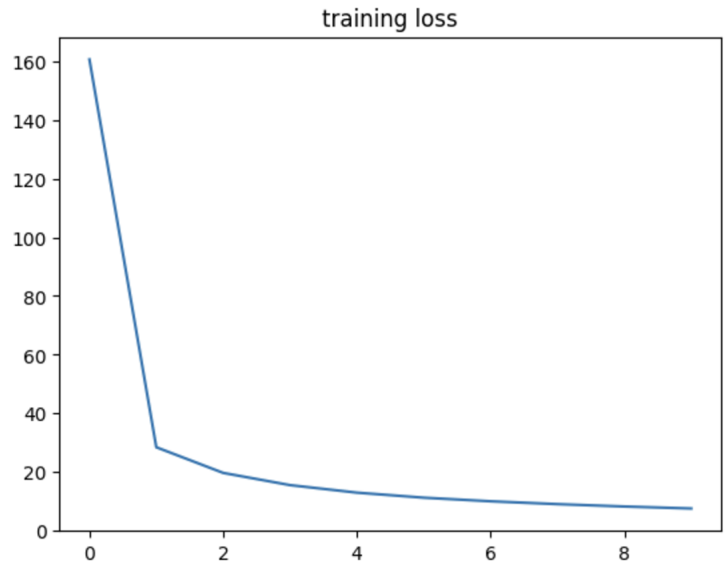

Figure 4: Training loss of the naive model training on the MNIST dataset

2.2 Domain Adverserial Neural Network

We will apply different changes on our model described on the FIGURE 5.

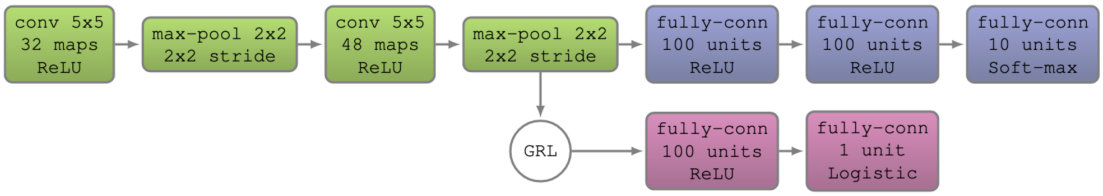

Figure 5: Architecture of the DANN model

In this new architecture, there is an additional branch. The purpose of this branch is to perform a binary classification between the MNIST domain and the MNIST-M

domain. However, the added GRL (Gradient Reversal Layer) allows for a "gradient reversal". Thus, to classify the data into the correct domain, the model must push the data as far away as possible from their domain before the gradient is reversed. Once reversed, the data are considered to belong to the correct domain. In this way, the features retained by the model will mainly correspond to the information related to the digit in the image, which is the common element between the MNIST and MNIST-M datasets.

To reverse the gradient, we use a GRL-factor (that is negative) and will perform the reversal. We will study the effect of his value in a following section but for now we assume that the value of this factor will follow the function described in FIGURE 6.

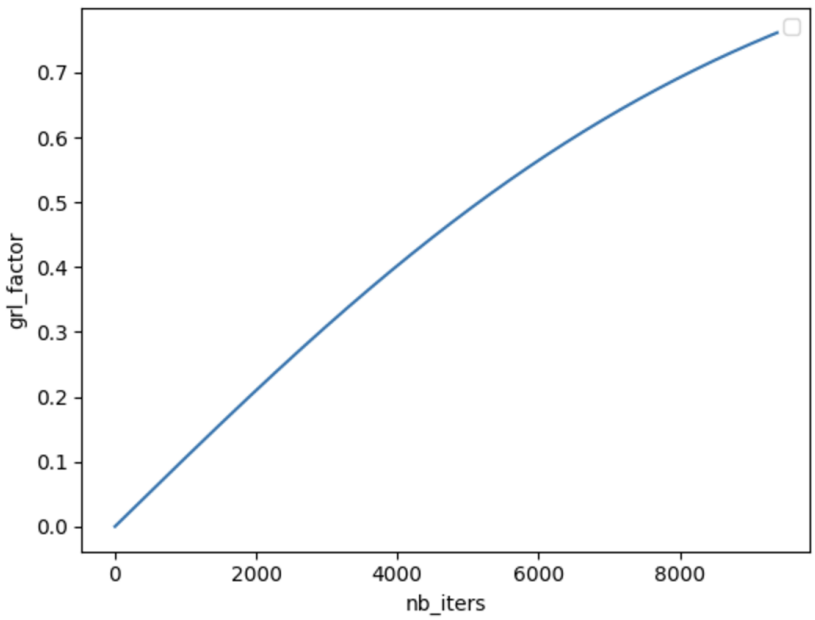

Figure 6: GRL factor value function (must be multiplied by -1)

Thanks to this new method, the performance reaches an accuracy of **76.03** on the MNIST-M dataset which is a considerable gain.

As we can see on the FIGURE 7, the embeddings of the two datasets after the naive CNN belong to different domains. Also, using the domain adaptation made the embeddings mix up so that the data is agnostic to its original domain.

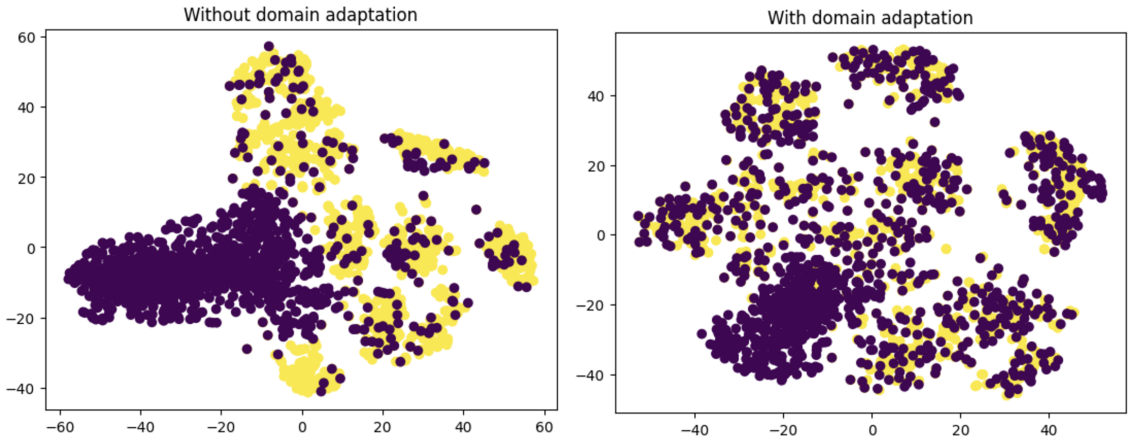

Figure 7: Visualization of the domain's spaces for the MNIST dataset (yellow dots) and MNIST-M dataset(purple dots)

Question 1

The domain classifier's gradient helps the model to separate features from the source and target domains. Without the GRL, this gradient would influence the feature extractor to focus on domain-specific features, making it easier for the domain classifier to tell the domains apart. However, this would cause the model to become domain-specific instead of learning features that work across both domains, which goes against the goal of domain adaptation. As expected, the model's performance goes down to an accuracy of **39.28** which is even lower than the naive model. This can be explained by the fact that the domain classification part makes the model focus on aspects of the image not related to the digit (like for example color) and makes the task more difficult for the model when it comes to find the right class for MNIST-M data (based on the digit).

Question 2

The source domain accuracy is always really high, from 99.07 at first to 98.76 when adding the domain adaptation and went up to 99.25 when I removed the GRL but kept the domain classification part. I think that the small decrease on this part can be explained by the fact that the domain adaptation part makes the model focus on other aspects of the image such as color and texture that are not present in the source dataset where the image are only digits in black and white. Therefore, the model focuses on irrelevant aspects of the image for black and white digit classification which makes it lose a very small bit of accuracy.

Question 3

The influence of the GRL factor is detailed in the subsection "GRL factor scheduling" in the next section.

Question 4

The pseudo-labeling method described in the article **[2]**, is a semi-supervised learning technique used in domain adaptation where a model is firstly trained on a labeled source domain. Then he generates "pseudo-labels" for the unlabeled target domain data. These pseudo-labels are then used to retrain the model where we use a combination of labeled data from the source domain with the pseudo-labeled data from the target domain. This process improves the model's performance on the target domain especially when labeled data is scarce. The method is iterative, with the model progressively refining its predictions using pseudo-labeled data. However, the quality of pseudo-labels depends on the initial model's accuracy, and incorrect pseudo-labels can introduce noise.

2.3 Model improvement

2.3.1 Learning rate scheduler

The aim of a learning rate scheduler is to maximize the speed and quality of the convergence, to keep the training stable and to help the generalization to the test dataset.

The Figure 8 shows that those 3 scheduler lead to pretty similar results. I chose the stepsize of the StepScheduler from 10 to 5 epochs since a stepsize of 10 made the model diverge. Indeed as we can see on Figure 9, choosing a step size that is too large can prevent the learning rate from decreasing for too long, which may lead to premature divergence of the model because the learning rate remains too high.

In the end, the first lr scheduler led to an accuracy of **77.73**, the second to **72.35** and the last to **77.68**.

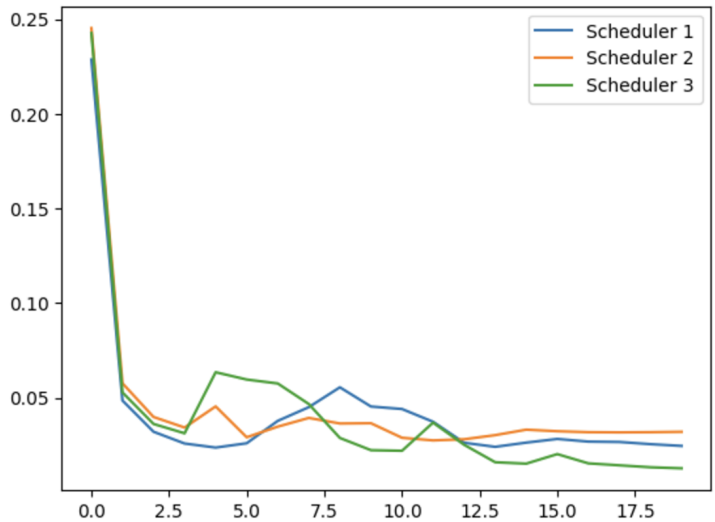

Figure 8: Training loss (class loss) of the DANN model for 3 different scheduler: (1) x -> 0.01 / (1 + alpha * x / epochs) ** beta), (2) StepScheduler, (3) Cosine Annealing Scheduler

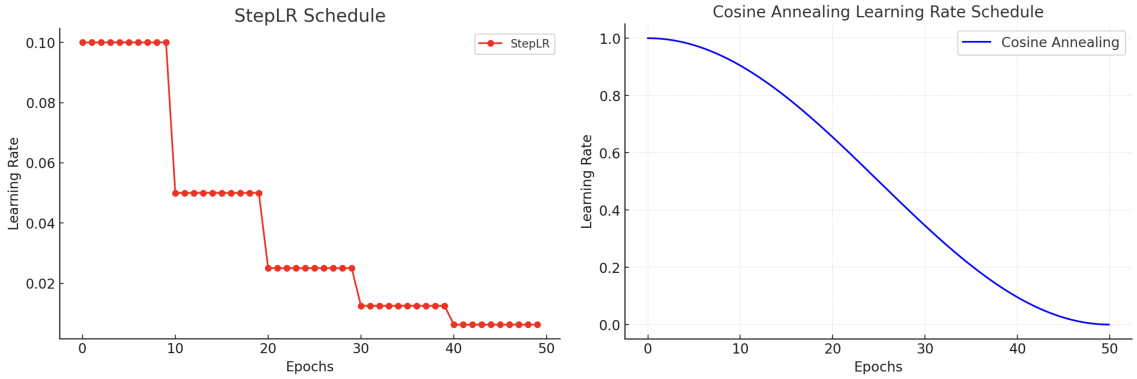

Figure 9: Comparison of StepLR scheduler's function (left) with Cosine Annealing Scheduler's function (right)

2.3.2 GRL factor scheduling

I tested 3 different functions as GRL factor scheduler in order to study the influence of its value on the DANN's performance.

The GRL factor controls the balance between source domain classification and domain adaptation. A low factor prioritizes source domain performance by reducing the influence of the domain classifier, limiting adaptation to the target domain.

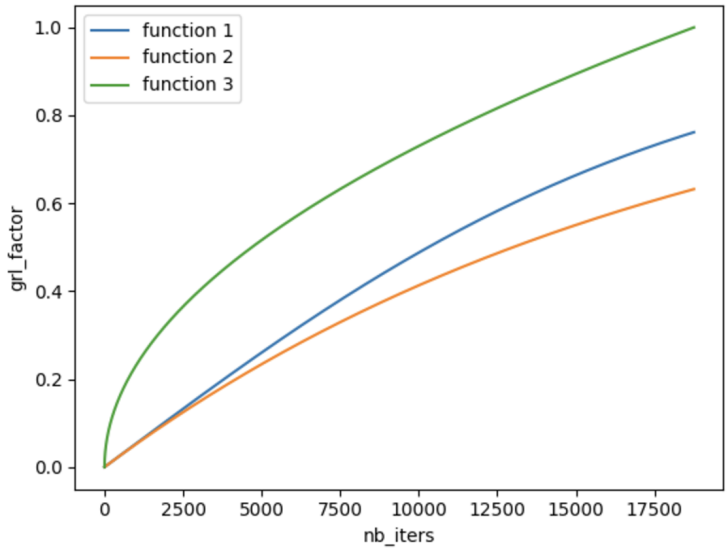

Figure 10: Representation of the different GRL factor scheduling functions: (1) $-1 + \frac{2}{1+e^{\frac{-2x}{\text{iters}}}}$ (2) $1 - e^{\frac{-x}{\text{iters}}}$ (3) $\sqrt{\frac{x}{\text{iters}}}$

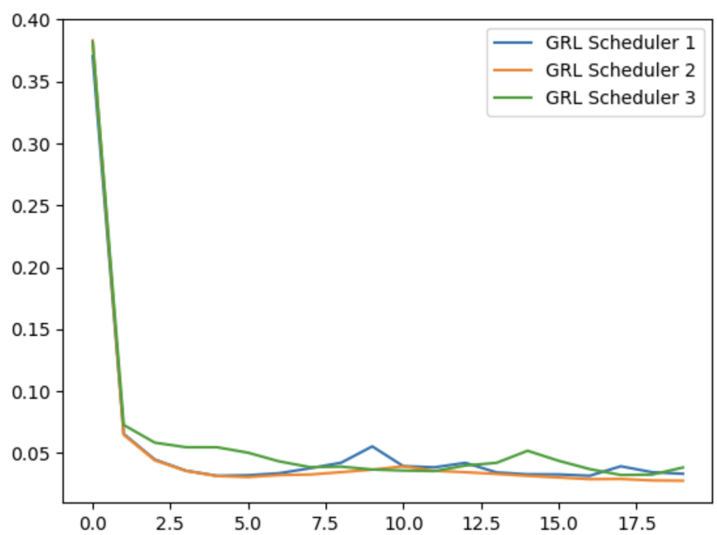

Figure 11: Training loss (class loss) for our 3 GRL factor scheduler

Conversely, a high factor forces the model to focus on learning domain-invariant features, which can improve adaptation but may harm source domain accuracy or cause instability. Indeed, the 3rd function that increases faster gave a bit lower

accuracy on the source domain (**97.89** against **98.78** for the function 2 and **98.80** for the function 1). However, the results on the target domain didn't increase and stayed between 73 and 78% for the three GRL factor scheduler.

2.3.3 Standardization

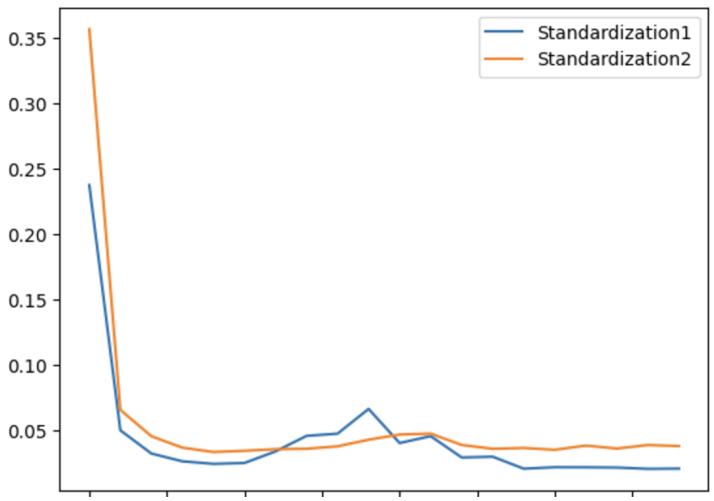

Figure 12: Training loss (class loss) of the DANN model for 2 different standardization of the datasets: (1) $\frac{x-\min}{\max-\min}$ and (2) $\frac{x-\sigma}{\nu}$

The use of min-max scaling appears to align well with CNN-type networks, as it achieves a slightly higher accuracy of **80.62** with this type of standardization. The changes in model performance remain relatively small, representing only a slight gain in precision.

2.3.4 Number of epochs

Finally, increasing the number of epochs slightly improves the model's accuracy, reaching **80.22**. However, accuracy stabilizes after about 50 epochs and remains around 80 even when extended to 100 epochs, making it computationally expensive without significant performance gains.

References

[1] Victor Lempitsky Yaroslav Ganin. *Unsupervised Domain Adaptation by Backpropagation*. URL: https://doi.org/10.48550/arXiv.1409.7495.

[2] Baoxin Li Sachin Chhabra Hemanth Venkateswara. *Domain Adaptation Using Pseudo Labels*. URL: https://doi.org/10.48550/arXiv.2402.06809.